

Así que, para confirmar el estado actual basándonos en tu corrección manual y aclaraciones:

Problema 1 (Orden de los Inlines): Resuelto (el orden es ahora `DetalleTransaccionProductoInline` seguido de `DetalleTransaccionInline`).

Problema 2 (Iconos de Acción Inline): El `DetalleTransaccionProductoForm` ya está en su lugar y asignado, y has añadido manualmente `self.fields['producto'].widget.can_view_related = False`. Esto significa que los iconos (lápiz, más, X, ojo) para el campo producto deberían estar ahora ocultos como se solicitó.

"el botón de eliminar detalle debes dejarlo, y el de `addDetalleProducto` también al momento de añadir una transacción.": Esto significa que la casilla de verificación para eliminar la fila inline y el botón "Añadir otro `DetalleTransaccionProducto`" deben ser visibles cuando se añade una nueva transacción. Los métodos `has_add_permission` y `has_delete_permission` que devuelven `obj is None` lo logran correctamente.

"Pero al momento de vista ya no se puede editar ningún campo de los registros ni detalles.": Esto significa que para transacciones existentes, todos los campos en `DetalleTransaccionProductoInline` deben ser de solo lectura. El método `DetalleTransaccionProductoInline.get_readonly_fields` actual lo maneja (`return [field.name for field in self.model._meta.fields if field.name != 'id']` cuando `obj is not None`).

Esto significa que el Problema 2 está ahora completamente resuelto, ¡gracias a tu intervención manual!

Ahora, procedamos con los Problemas 3 y 4 (que están interconectados):

Problema 3: El precio unitario no se establece en el Inline (cuando se selecciona un producto, `precio_unitario_venta` no se rellena automáticamente).

Problema 4: Falta el Total para los Productos dentro del Inline (Sub-Total Display) (No hay un subtotal visual para varias líneas de producto, y `sub_total` no se actualiza dinámicamente).

Estrategia para los Problemas 3 y 4 (JavaScript y Contexto del Administrador):

Obtener Precios de Productos (Servidor para JS):

Acción: Añadir un método a `TransaccionAdmin` (por ejemplo, sobrescribiendo `change_view`) para obtener todos los objetos `Producto` y su `precio_venta`.

Pasar estos datos (por ejemplo, como un diccionario que mapee el ID del producto al precio) al contexto de la plantilla.

Acción: Modificar `tesoreria/admin.py` para lograr esto.

Incrustar Precios de Productos en HTML:

Acción: Dado que `TransaccionAdmin` utiliza `admin/miembros/change_form.html`, y esa plantilla extiende `admin/change_form.html`, necesitamos una forma de inyectar nuestro

product_prices_json en una variable JavaScript a la que tesoreria/js/transaccion_total.js pueda acceder.

La forma más limpia es crear una plantilla change_form.html personalizada específicamente para tesoreria/transaccion (por ejemplo, admin/tesoreria/transaccion/change_form.html) que extienda la base e incruste los datos JSON.

Actualizar tesoreria/static/tesoreria/js/transaccion_total.js:

Acceder a Precios de Productos: Recuperar los precios de los productos del JSON incrustado.

Escuchar cambios en la selección de producto: Cuando se selecciona un producto en el inline, usar el ID del producto para buscar su precio_venta.

Populate precio_unitario_venta: Establecer el precio obtenido en el campo precio_unitario_venta para esa fila.

Escuchar cambios en cantidad y precio_unitario_venta: Cuando cualquiera de los dos cambie, calcular cantidad * precio_unitario_venta y actualizar el campo sub_total para esa fila.

Actualizar Monto Total General: Asegurarse de que monto_total_visual_id principal se actualice correctamente sumando todos los sub_total de los inlines de productos y los monto de los inlines de pago.